Notice how this function is assigned to the variable `add`? That's an important detail that underpins most of this chapter. You see, functions don't just produce values; they *are* values. This means that a function can produce a function as its return value and take a function as an argument. This idea that functions are values is referred to as functions being *first-class* and leads to the powerful technique of gluing functions together, called *function composition*.

How does this relate to JavaScript? Functions are just as powerful in JavaScript, but unfortunately functions in JavaScript have some problems. CoffeeScript aims to address these problems and improve the syntax. In this chapter you'll learn how to use functions for events, callbacks, and higher-order programming and why CoffeeScript syntax is a clearer way to describe functions than the equivalent JavaScript.

3.1. Computation

Imagine your friends Agtron and Scruffy are having a party, and you've been tasked with writing a program to count the number of confirmed attendees. When the program counts the attendees for you, it performs a *computation*. Think of functions as being little computers—they perform computations. To grasp this metaphor, you need to start with some basics.

3.1.1. Basics

You can multiply the numbers 3 and 4 using the multiplication operator:

```
3 * 4
# 12
```

Here's a function to perform the same operation:

```
threeTimesFour = -> 3 * 4
# [Function]
```

Declaring a function literal evaluates to a function, shown on the REPL as [Function]